Testing con StepVerifier

Fundamentos StepVerifier

- La clase reactor.test.StepVerifier proporciona un medio para verificar una secuencia asíncrona generada por un publicador (Mono o Flux)

- Se basa en realizar suposiciones acerca de los eventos que tendrán lugar tras una suscripción.

- La verificación se lanza tras completarse la operación, mediante alguno de los método *verify()*

- Además del starter-test, se requiere la dependencia reactor-test:

```
<dependency>
    <groupId>io.projectreactor</groupId>
    <artifactId>reactor-test</artifactId>
</dependency>
```

Antonio Martín Sierra

Preparación

- A través del método estático *create()* de StepVerifier preparamos el objeto para comenzar con las pruebas

- Se le proporciona al método el objeto Publisher que queremos probar.

```
Flux<Integer> flux=Flux.just(10,3,4);
StepVerifier.create(flux)//FirstStep
```

- La llamada a *create()* funciona a modo de Builder, devolviendo un objeto FirstStep sobre el que se realizarán llamadas a métodos *expectNext()* para configurar las suposiciones de cada evento

Antonio Martínez Sierra

Suposiciones I

- **Para realizar las suposiciones sobre los datos generados en cada evento, se van realizando llamadas a Step *expectNext(T... elements)* :**

```
Flux<Integer> flux=Flux.just(10,3,4);
StepVerifier.create(flux)//FirstStep
   .expectNext(10) //Step
   .expectNext(3)  //Step
```

- **Mediate expectNextCount(long count) se puede indicar que se espera recibir una serie de elementos después del último evento:**

```
Flux<Integer> flux=Flux.just(10,3,4);
StepVerifier.create(flux)//FirstStep
   .expectNext(10) //Step
   .expectNextCount(2)  //Step. Se espera recibir dos
                        //números después del 10
```

Suposiciones II

- En lugar de comprobar la coincidencia con un elemento, se puede establecer un predicado a cumplir mediante el método expectNextMatches() :

```
Flux<Persona> flux=Flux.just(new Persona("a",25),new Persona("b",43), new Persona("b",43));
StepVerifier.create(flux)//FirstStep
   .expectNextMatches(p->p.getName().equals("a")) //Step
   .expectNextCount(2)  //Step
```

- Otros métodos :
  - Step then(Runnable rn). Tras la suposición anterior, ejecuta el Runnable
  - StepVerifier expectComplete(). Espera que después de la suposición anterior llegue la señal de finalización

Antonio Martín

Verificación

- **Tras completar las suposiciones, se realiza la llamada al método *verifyComplete()* para verificar el proceso:**

```
Flux<Integer> flux=Flux.just(10,3,4);
StepVerifier.create(flux)//FirstStep
    .expectNext(10) //Step
    .expectNext(3,4)  //Step
    .verifyComplete();
```

- **Se puede realizar la verificación sin necesidad de completar las suposiciones, llamando previamente a *thenCancel()*:**

```
Flux<Integer> flux=Flux.just(10,3,4);
StepVerifier.create(flux)//FirstStep
    .expectNext(10) //Step
    .thenCancel()  //StepVerifier
    .verify ();
```

Antonio Martín